Code Explanation

1. System Overview

This system is a Campus Library Management System that supports two roles: **Admin** and **Student**. The backend provides REST APIs and serves the frontend pages, while SQLite stores all library data.

High-level working (block diagram):

```
server.js
 |
 v
backend/server.js (Express App)
 |
 +-- Serves static frontend (public/)
 +-- Registers REST API routes (/api/*)
 +-- Loads environment variables (.env)
 +-- Initializes SQLite database (backend/config/db.js)
 |
 +-- Routes (backend/routes/*)
 |    +-- authRoutes  -> authController
 |    +-- bookRoutes  -> bookController
 |    +-- userRoutes  -> userController
 |    `-- issueRoutes -> issueController
 |
 +-- Middleware (backend/middleware/*)
 |    +-- authMiddleware (JWT verify)
 |    `-- roleMiddleware (RBAC)
 |
 `-- Database Layer (backend/database/*)
      +-- schema.sql
      +-- seed.sql
      `-- library.db
```

3. Architecture

- **Backend (Node.js + Express):** Handles authentication, book management, student management, and issue/return operations using REST APIs.
- **Database (SQLite):** Stores users, books, and issue records. Database is initialized on server start using schema and seed scripts.
- **Frontend (Static HTML/CSS/JS):** Pages in public/ call backend APIs using JavaScript (fetch) and show data for Admin/Student dashboards.

- **Security:** JWT-based authentication; role-based access control ensures only Admin can manage books/students and issue/return books.

4. Updated Folder Structure

```
.
├── server.js
├── package.json
├── backend
│   ├── server.js
│   ├── config
│   │   └── db.js
│   ├── controllers
│   │   ├── authController.js
│   │   ├── bookController.js
│   │   ├── issueController.js
│   │   └── userController.js
│   ├── middleware
│   │   ├── authMiddleware.js
│   │   └── roleMiddleware.js
│   ├── routes
│   │   ├── authRoutes.js
│   │   ├── bookRoutes.js
│   │   ├── issueRoutes.js
│   │   └── userRoutes.js
│   ├── utils
│   │   └── generateToken.js
│   └── database
│       ├── schema.sql
│       ├── seed.sql
│       └── library.db
└── public
    ├── *.html
    ├── css
    │   └── *.css
    ├── js
    │   └── *.js
    └── assets
        └── logo.png
```

5. File-by-File Explanation

File: server.js

Description: Root project entrypoint used by local run and Render deployment.

Code:

```
require("./backend/server");
```

File: package.json

Description: Stores project metadata, active scripts, and deployment dependencies for the root project.

Code:

```
{
  "name": "campus-library-management-system",
  "version": "1.0.0",
  "description": "Simple BCA third-year level library management system",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.21.2",
    "jsonwebtoken": "^9.0.2",
    "sqlite3": "^5.1.7"
  },
  "devDependencies": {
    "nodemon": "^3.1.10"
  }
}
```

File: backend/server.js

Description: Main server file that configures Express, serves the `public` frontend, exposes API routes, and starts the server after database initialization.

Code:

```
const path = require("path");
const express = require("express");
const cors = require("cors");
const dotenv = require("dotenv");
const { initializeDatabase } = require("./config/db");

dotenv.config();

const authRoutes = require("./routes/authRoutes");
const bookRoutes = require("./routes/bookRoutes");
const userRoutes = require("./routes/userRoutes");
```

```
const issueRoutes = require("./routes/issueRoutes");

const app = express();
const publicPath = path.join(__dirname, "..", "public");
const staticPath = path.resolve(publicPath);

app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.get("/", (req, res) => {
  res.sendFile(path.join(staticPath, "index.html"));
});

app.use(express.static(staticPath));

app.use("/api/auth", authRoutes);
app.use("/api/books", bookRoutes);
app.use("/api/users", userRoutes);
app.use("/api/issues", issueRoutes);

app.get("/api/health", (request, response) => {
  response.json({
    success: true,
    message: "Server is running"
  });
});

app.use((request, response) => {
  response.status(404).json({
    success: false,
    message: "Route not found"
  });
});

const PORT = process.env.PORT || 5000;

initializeDatabase()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
    });
  })
  .catch((error) => {
    console.error("Database initialization failed", error);
    process.exit(1);
  });
```

File: backend/config/db.js

Description: Database configuration file that creates the SQLite connection, supports environment-based database paths, and runs schema initialization plus seed data setup.

Code:

```
const fs = require("fs");
const path = require("path");
const sqlite3 = require("sqlite3").verbose();
const bcrypt = require("bcryptjs");

const appDatabaseDirectory = path.join(__dirname, "..", "database");
const configuredDatabasePath = process.env.DB_PATH
  ? path.resolve(process.env.DB_PATH)
  : path.join(appDatabaseDirectory, "library.db");
const databaseDirectory = path.dirname(configuredDatabasePath);
const databasePath = configuredDatabasePath;
const schemaPath = path.join(appDatabaseDirectory, "schema.sql");
const seedPath = path.join(appDatabaseDirectory, "seed.sql");

fs.mkdirSync(databaseDirectory, { recursive: true });

const db = new sqlite3.Database(databasePath);

function run(query, params = []) {
  return new Promise((resolve, reject) => {
    db.run(query, params, function runCallback(error) {
      if (error) {
        reject(error);
        return;
      }

      resolve(this);
    });
  });
}

function get(query, params = []) {
  return new Promise((resolve, reject) => {
    db.get(query, params, (error, row) => {
      if (error) {
        reject(error);
        return;
      }

      resolve(row);
    });
```

```
  });
}

function all(query, params = []) {
  return new Promise((resolve, reject) => {
    db.all(query, params, (error, rows) => {
      if (error) {
        reject(error);
        return;
      }

      resolve(rows);
    });
  });
}

function exec(sql) {
  return new Promise((resolve, reject) => {
    db.exec(sql, (error) => {
      if (error) {
        reject(error);
        return;
      }

      resolve();
    });
  });
}

async function seedAdmin() {
  const adminUser = await get(
    "SELECT id FROM users WHERE email = ? LIMIT 1",
    ["admin@library.com"]
  );

  if (adminUser) {
    return;
  }

  const seedSql = fs.readFileSync(seedPath, "utf8");
  const adminPasswordHash = bcrypt.hashSync("admin123", 10);
  const finalSeedSql = seedSql.replace("{{ADMIN_PASSWORD_HASH}}",
adminPasswordHash);

  await exec(finalSeedSql);
}

async function seedSampleData() {
  const sampleStudentPassword = bcrypt.hashSync("student123", 10);
```

```
const sampleStudents = [
  {
    full_name: "Aman Sharma",
    email: "aman.sharma@example.com",
    roll_number: "BCA2023001",
    course: "BCA",
    phone_number: "9876543210"
  },
  {
    full_name: "Priya Verma",
    email: "priya.verma@example.com",
    roll_number: "BCA2023002",
    course: "BCA",
    phone_number: "9812345678"
  },
  {
    full_name: "Rohit Singh",
    email: "rohit.singh@example.com",
    roll_number: "BCA2023003",
    course: "BCA",
    phone_number: "9898989898"
  },
  {
    full_name: "Neha Gupta",
    email: "neha.gupta@example.com",
    roll_number: "BCA2023004",
    course: "BCA",
    phone_number: "9823456789"
  }
];

const sampleBooks = [
  {
    title: "Introduction to C Programming",
    author: "E. Balagurusamy",
    category: "Programming",
    isbn: "9789388176055",
    total_copies: 5,
    shelf_location: "Rack A-1",
    description: "Basic programming concepts for beginners."
  },
  {
    title: "Database System Concepts",
    author: "Abraham Silberschatz",
    category: "Database",
    isbn: "9780073523323",
    total_copies: 4,
    shelf_location: "Rack B-2",
    description: "Covers relational databases, SQL, and transactions."
```

```
  },
  {
    title: "Computer Networks",
    author: "Andrew S. Tanenbaum",
    category: "Networking",
    isbn: "9789332585490",
    total_copies: 3,
    shelf_location: "Rack C-1",
    description: "Simple and practical explanation of network concepts."
  },
  {
    title: "Operating System Principles",
    author: "Abraham Silberschatz",
    category: "Operating System",
    isbn: "9788126551960",
    total_copies: 4,
    shelf_location: "Rack C-3",
    description: "Helps students understand processes, memory, and files."
  },
  {
    title: "Web Technologies",
    author: "Uttam K. Roy",
    category: "Web Development",
    isbn: "9780199464500",
    total_copies: 6,
    shelf_location: "Rack D-1",
    description: "Useful for HTML, CSS, JavaScript, and web basics."
  }
];

for (const student of sampleStudents) {
  await run(
    `INSERT OR IGNORE INTO users
    (full_name, email, password, role, roll_number, course, phone_number)
    VALUES (?, ?, ?, 'student', ?, ?, ?)`,
    [
      student.full_name,
      student.email,
      sampleStudentPassword,
      student.roll_number,
      student.course,
      student.phone_number
    ]
  );
}

for (const book of sampleBooks) {
  await run(
    `INSERT OR IGNORE INTO books
    (title, author, category, isbn, total_copies, available_copies,
```

```
shelf_location, description)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?)`,
      [
        book.title,
        book.author,
        book.category,
        book.isbn,
        book.total_copies,
        book.total_copies,
        book.shelf_location,
        book.description
      ]
    );
  }

  const sampleIssues = [
    {
      studentEmail: "aman.sharma@example.com",
      bookIsbn: "9789388176055",
      issue_date: "2026-04-01",
      due_date: "2026-04-15",
      return_date: null,
      status: "issued"
    },
    {
      studentEmail: "priya.verma@example.com",
      bookIsbn: "9780073523323",
      issue_date: "2026-03-20",
      due_date: "2026-04-03",
      return_date: null,
      status: "issued"
    },
    {
      studentEmail: "aman.sharma@example.com",
      bookIsbn: "9780199464500",
      issue_date: "2026-03-10",
      due_date: "2026-03-24",
      return_date: "2026-03-22",
      status: "returned"
    }
  ];

  for (const issue of sampleIssues) {
    const student = await get("SELECT id FROM users WHERE email = ? LIMIT 1",
[
      issue.studentEmail
    ]);
    const book = await get("SELECT id FROM books WHERE isbn = ? LIMIT 1", [
      issue.bookIsbn
    ]);
```

```
    if (!student || !book) {
      continue;
    }

    const existingIssue = await get(
      `SELECT id FROM issue_records
       WHERE student_id = ? AND book_id = ? AND issue_date = ? AND due_date =
?
       LIMIT 1`,
      [student.id, book.id, issue.issue_date, issue.due_date]
    );

    if (existingIssue) {
      continue;
    }

    await run(
      `INSERT INTO issue_records
      (student_id, book_id, issue_date, due_date, return_date, status)
      VALUES (?, ?, ?, ?, ?, ?)`,
      [
        student.id,
        book.id,
        issue.issue_date,
        issue.due_date,
        issue.return_date,
        issue.status
      ]
    );

    if (issue.status === "issued") {
      await run(
        `UPDATE books
         SET available_copies = available_copies - 1, updated_at =
CURRENT_TIMESTAMP
         WHERE id = ? AND available_copies > 0`,
        [book.id]
      );
    }
  }
}

async function initializeDatabase() {
  const schemaSql = fs.readFileSync(schemaPath, "utf8");
  await exec(schemaSql);
  await seedAdmin();
  await seedSampleData();
}
```

```
module.exports = {
  db,
  run,
  get,
  all,
  exec,
  initializeDatabase
};
```

File: backend/utils/generateToken.js

Description: Generates JWT tokens after successful authentication.

Code:

```
const jwt = require("jsonwebtoken");

function generateToken(user) {
  return jwt.sign(
    {
      id: user.id,
      email: user.email,
      role: user.role
    },
    process.env.JWT_SECRET,
    {
      expiresIn: "1d"
    }
  );
}

module.exports = generateToken;
```

File: backend/middleware/authMiddleware.js

Description: Verifies JWT tokens and protects private routes by attaching the authenticated user to the request.

Code:

```
const jwt = require("jsonwebtoken");
const { get } = require("../config/db");

async function authMiddleware(request, response, next) {
  try {
    const authorizationHeader = request.headers.authorization;

    if (!authorizationHeader || !authorizationHeader.startsWith("Bearer ")) {
      return response.status(401).json({
```

```
        success: false,
        message: "Unauthorized access"
      });
    }

    const token = authorizationHeader.split(" ")[1];
    const decodedToken = jwt.verify(token, process.env.JWT_SECRET);
    const user = await get(
      "SELECT id, full_name, email, role, roll_number, course, phone_number,
created_at, updated_at FROM users WHERE id = ?",
      [decodedToken.id]
    );

    if (!user) {
      return response.status(401).json({
        success: false,
        message: "User not found"
      });
    }

    request.user = user;
    next();
  } catch (error) {
    return response.status(401).json({
      success: false,
      message: "Invalid or expired token"
    });
  }
}

module.exports = authMiddleware;
```

File: backend/middleware/roleMiddleware.js

Description: Restricts route access based on user role such as admin or student.

Code:

```
function roleMiddleware(...allowedRoles) {
  return function checkRole(request, response, next) {
    if (!request.user || !allowedRoles.includes(request.user.role)) {
      return response.status(403).json({
        success: false,
        message: "Unauthorized access"
      });
    }

    next();
  };
}
```

```
module.exports = roleMiddleware;
```

File: backend/controllers/authController.js

Description: Handles student registration, user login, and returning details of the currently logged-in user.

Code:

```
const bcrypt = require("bcryptjs");
const { get, run } = require("../config/db");
const generateToken = require("../utils/generateToken");

async function registerStudent(request, response) {
  try {
    const {
      full_name,
      email,
      password,
      roll_number,
      course,
      phone_number
    } = request.body;

    if (!full_name || !email || !password || !roll_number || !course ||
!phone_number) {
      return response.status(400).json({
        success: false,
        message: "All fields are required"
      });
    }

    if (password.length < 6) {
      return response.status(400).json({
        success: false,
        message: "Password must be at least 6 characters"
      });
    }

    const existingUser = await get(
      "SELECT id FROM users WHERE email = ? OR roll_number = ? LIMIT 1",
      [email.trim(), roll_number.trim()]
    );

    if (existingUser) {
      return response.status(400).json({
        success: false,
        message: "Email or roll number already exists"
      });
```

```
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    await run(
      `INSERT INTO users (full_name, email, password, role, roll_number,
course, phone_number)
       VALUES (?, ?, ?, 'student', ?, ?, ?)`,
      [
        full_name.trim(),
        email.trim().toLowerCase(),
        hashedPassword,
        roll_number.trim(),
        course.trim(),
        phone_number.trim()
      ]
    );

    return response.status(201).json({
      success: true,
      message: "Student registered successfully"
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to register student"
    });
  }
}

async function loginUser(request, response) {
  try {
    const { email, password } = request.body;

    if (!email || !password) {
      return response.status(400).json({
        success: false,
        message: "Email and password are required"
      });
    }

    const user = await get("SELECT * FROM users WHERE email = ? LIMIT 1", [
      email.trim().toLowerCase()
    ]);

    if (!user) {
      return response.status(401).json({
        success: false,
        message: "Invalid login credentials"
```

```
      });
    }

    const isPasswordCorrect = await bcrypt.compare(password, user.password);

    if (!isPasswordCorrect) {
      return response.status(401).json({
        success: false,
        message: "Invalid login credentials"
      });
    }

    const token = generateToken(user);

    return response.json({
      success: true,
      message: "Login successful",
      token,
      user: {
        id: user.id,
        full_name: user.full_name,
        email: user.email,
        role: user.role,
        roll_number: user.roll_number,
        course: user.course,
        phone_number: user.phone_number
      }
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to login"
    });
  }
}

function getCurrentUser(request, response) {
  return response.json({
    success: true,
    user: request.user
  });
}

module.exports = {
  registerStudent,
  loginUser,
  getCurrentUser
};
```

File: backend/controllers/bookController.js

Description: Implements book-related operations such as listing, searching, adding, updating, and deleting books.

Code:

```
const { all, get, run } = require("../config/db");

function buildBookStatus(book) {
  return {
    ...book,
    status: Number(book.available_copies) > 0 ? "Available" : "Not Available"
  };
}

async function getBooks(request, response) {
  try {
    const search = request.query.search ? request.query.search.trim() : "";
    const category = request.query.category ? request.query.category.trim() :
"";

    let query = "SELECT * FROM books WHERE 1 = 1";
    const params = [];

    if (search) {
      query += " AND (title LIKE ? OR author LIKE ?)";
      params.push(`%${search}%`, `%${search}%`);
    }

    if (category) {
      query += " AND category = ?";
      params.push(category);
    }

    query += " ORDER BY created_at DESC";

    const books = await all(query, params);

    return response.json({
      success: true,
      books: books.map(buildBookStatus)
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to fetch books"
    });
  }
```

```
}

async function getBookById(request, response) {
  try {
    const bookId = Number(request.params.id);

    if (!bookId) {
      return response.status(400).json({
        success: false,
        message: "Invalid book id"
      });
    }

    const book = await get("SELECT * FROM books WHERE id = ?", [bookId]);

    if (!book) {
      return response.status(404).json({
        success: false,
        message: "Book not found"
      });
    }

    return response.json({
      success: true,
      book: buildBookStatus(book)
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to fetch book"
    });
  }
}

async function addBook(request, response) {
  try {
    const {
      title,
      author,
      category,
      isbn,
      total_copies,
      shelf_location,
      description
    } = request.body;

    const totalCopies = Number(total_copies);

    if (!title || !author || !category || !isbn || !totalCopies) {
```

```
      return response.status(400).json({
        success: false,
        message: "Required fields are missing"
      });
    }

    if (totalCopies < 1) {
      return response.status(400).json({
        success: false,
        message: "Total copies must be at least 1"
      });
    }

    const existingBook = await get("SELECT id FROM books WHERE isbn = ? LIMIT
1", [
      isbn.trim()
    ]);

    if (existingBook) {
      return response.status(400).json({
        success: false,
        message: "ISBN already exists"
      });
    }

    await run(
      `INSERT INTO books
      (title, author, category, isbn, total_copies, available_copies,
shelf_location, description)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?)`,
      [
        title.trim(),
        author.trim(),
        category.trim(),
        isbn.trim(),
        totalCopies,
        totalCopies,
        shelf_location ? shelf_location.trim() : "",
        description ? description.trim() : ""
      ]
    );

    return response.status(201).json({
      success: true,
      message: "Book added successfully"
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
```

```
      message: "Failed to add book"
    });
  }
}

async function updateBook(request, response) {
  try {
    const bookId = Number(request.params.id);
    const {
      title,
      author,
      category,
      total_copies,
      shelf_location,
      description
    } = request.body;

    if (!bookId) {
      return response.status(400).json({
        success: false,
        message: "Invalid book id"
      });
    }

    if (!title || !author || !category || !total_copies) {
      return response.status(400).json({
        success: false,
        message: "Required fields are missing"
      });
    }

    const totalCopies = Number(total_copies);

    if (totalCopies < 1) {
      return response.status(400).json({
        success: false,
        message: "Total copies must be at least 1"
      });
    }

    const book = await get("SELECT * FROM books WHERE id = ?", [bookId]);

    if (!book) {
      return response.status(404).json({
        success: false,
        message: "Book not found"
      });
    }
```

```
    const issuedCopies = Number(book.total_copies) -
Number(book.available_copies);

    if (totalCopies < issuedCopies) {
      return response.status(400).json({
        success: false,
        message: "Total copies cannot be less than currently issued copies"
      });
    }

    const availableCopies = totalCopies - issuedCopies;

    await run(
      `UPDATE books
       SET title = ?, author = ?, category = ?, total_copies = ?,
available_copies = ?,
           shelf_location = ?, description = ?, updated_at =
CURRENT_TIMESTAMP
       WHERE id = ?`,
      [
        title.trim(),
        author.trim(),
        category.trim(),
        totalCopies,
        availableCopies,
        shelf_location ? shelf_location.trim() : "",
        description ? description.trim() : "",
        bookId
      ]
    );

    return response.json({
      success: true,
      message: "Book updated successfully"
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to update book"
    });
  }
}

async function deleteBook(request, response) {
  try {
    const bookId = Number(request.params.id);

    if (!bookId) {
      return response.status(400).json({
```

```
        success: false,
        message: "Invalid book id"
      });
    }

    const book = await get("SELECT id FROM books WHERE id = ?", [bookId]);

    if (!book) {
      return response.status(404).json({
        success: false,
        message: "Book not found"
      });
    }

    const activeIssue = await get(
      "SELECT id FROM issue_records WHERE book_id = ? AND status = 'issued'
LIMIT 1",
      [bookId]
    );

    if (activeIssue) {
      return response.status(400).json({
        success: false,
        message: "Book cannot be deleted because it is currently issued"
      });
    }

    await run("DELETE FROM books WHERE id = ?", [bookId]);

    return response.json({
      success: true,
      message: "Book deleted successfully"
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to delete book"
    });
  }
}

module.exports = {
  getBooks,
  getBookById,
  addBook,
  updateBook,
  deleteBook
};
```

File: backend/controllers/userController.js

Description: Handles student list, student details, profile update, and student deletion functions.

Code:

```
const { all, get, run } = require("../config/db");

async function getStudents(request, response) {
  try {
    const students = await all(
      `SELECT id, full_name, email, roll_number, course, phone_number,
'active' AS status, created_at
       FROM users
       WHERE role = 'student'
       ORDER BY created_at DESC`
    );

    return response.json({
      success: true,
      students
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to fetch students"
    });
  }
}

async function getStudentById(request, response) {
  try {
    const studentId = Number(request.params.id);

    if (!studentId) {
      return response.status(400).json({
        success: false,
        message: "Invalid student id"
      });
    }

    const student = await get(
      `SELECT id, full_name, email, roll_number, course, phone_number,
created_at, updated_at
       FROM users
       WHERE id = ? AND role = 'student'`,
      [studentId]
    );
```

```
    if (!student) {
      return response.status(404).json({
        success: false,
        message: "Student not found"
      });
    }

    const issueRecords = await all(
      `SELECT issue_records.id, books.title, issue_records.issue_date,
issue_records.due_date,
              issue_records.return_date, issue_records.status
       FROM issue_records
       INNER JOIN books ON books.id = issue_records.book_id
       WHERE issue_records.student_id = ?
       ORDER BY issue_records.created_at DESC`,
      [studentId]
    );

    return response.json({
      success: true,
      student,
      issueRecords
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to fetch student details"
    });
  }
}

async function updateProfile(request, response) {
  try {
    const { full_name, course, phone_number } = request.body;

    if (!full_name || !course || !phone_number) {
      return response.status(400).json({
        success: false,
        message: "All fields are required"
      });
    }

    await run(
      `UPDATE users
       SET full_name = ?, course = ?, phone_number = ?, updated_at =
CURRENT_TIMESTAMP
       WHERE id = ?`,
      [full_name.trim(), course.trim(), phone_number.trim(), request.user.id]
```

```
    );

    const updatedUser = await get(
      `SELECT id, full_name, email, role, roll_number, course, phone_number,
created_at, updated_at
       FROM users WHERE id = ?`,
      [request.user.id]
    );

    return response.json({
      success: true,
      message: "Profile updated successfully",
      user: updatedUser
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to update profile"
    });
  }
}

async function deleteStudent(request, response) {
  try {
    const studentId = Number(request.params.id);

    if (!studentId) {
      return response.status(400).json({
        success: false,
        message: "Invalid student id"
      });
    }

    const student = await get(
      "SELECT id FROM users WHERE id = ? AND role = 'student'",
      [studentId]
    );

    if (!student) {
      return response.status(404).json({
        success: false,
        message: "Student not found"
      });
    }

    const activeIssue = await get(
      "SELECT id FROM issue_records WHERE student_id = ? AND status =
'issued' LIMIT 1",
      [studentId]
```

```
    );

    if (activeIssue) {
      return response.status(400).json({
        success: false,
        message: "Student cannot be deleted because active issued books
exist"
      });
    }

    await run("DELETE FROM users WHERE id = ? AND role = 'student'",
[studentId]);

    return response.json({
      success: true,
      message: "Student deleted successfully"
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to delete student"
    });
  }
}

module.exports = {
  getStudents,
  getStudentById,
  updateProfile,
  deleteStudent
};
```

File: backend/controllers/issueController.js

Description: Manages issue, return, dashboard summary, and issued-book record operations.

Code:

```
const { all, get, run } = require("../config/db");

function isOverdue(issueDate, status) {
  if (status !== "issued") {
    return false;
  }

  return new Date().toISOString().split("T")[0] > issueDate;
}
```

```
function addOverdueFlag(record) {
  return {
    ...record,
    overdue: isOverdue(record.due_date, record.status)
  };
}

async function createIssue(request, response) {
  try {
    const { student_id, book_id, due_date } = request.body;
    const studentId = Number(student_id);
    const bookId = Number(book_id);
    const today = new Date().toISOString().split("T")[0];

    if (!studentId || !bookId || !due_date) {
      return response.status(400).json({
        success: false,
        message: "Student, book and due date are required"
      });
    }

    if (due_date < today) {
      return response.status(400).json({
        success: false,
        message: "Due date should not be before today"
      });
    }

    const student = await get(
      "SELECT id FROM users WHERE id = ? AND role = 'student'",
      [studentId]
    );
    const book = await get("SELECT * FROM books WHERE id = ?", [bookId]);

    if (!student) {
      return response.status(404).json({
        success: false,
        message: "Student not found"
      });
    }

    if (!book) {
      return response.status(404).json({
        success: false,
        message: "Book not found"
      });
    }

    if (Number(book.available_copies) < 1) {
```

```
      return response.status(400).json({
        success: false,
        message: "Selected book is not available"
      });
    }

    const duplicateIssue = await get(
      `SELECT id FROM issue_records
       WHERE student_id = ? AND book_id = ? AND status = 'issued'
       LIMIT 1`,
      [studentId, bookId]
    );

    if (duplicateIssue) {
      return response.status(400).json({
        success: false,
        message: "This student already has this book issued"
      });
    }

    await run(
      `INSERT INTO issue_records (student_id, book_id, issue_date, due_date,
status)
       VALUES (?, ?, ?, ?, 'issued')`,
      [studentId, bookId, today, due_date]
    );

    await run(
      `UPDATE books
       SET available_copies = available_copies - 1, updated_at =
CURRENT_TIMESTAMP
       WHERE id = ?`,
      [bookId]
    );

    return response.status(201).json({
      success: true,
      message: "Book issued successfully"
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to issue book"
    });
  }
}

async function getIssues(request, response) {
  try {
```

```
    const issues = await all(
      `SELECT issue_records.id, users.full_name AS student_name,
users.roll_number,
              books.title AS book_title, issue_records.issue_date,
issue_records.due_date,
              issue_records.return_date, issue_records.status
       FROM issue_records
       INNER JOIN users ON users.id = issue_records.student_id
       INNER JOIN books ON books.id = issue_records.book_id
       ORDER BY issue_records.created_at DESC`
    );

    return response.json({
      success: true,
      issues: issues.map(addOverdueFlag)
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to fetch issue records"
    });
  }
}

async function getMyIssues(request, response) {
  try {
    const issues = await all(
      `SELECT issue_records.id, books.title AS book_title, books.author,
books.category,
              issue_records.issue_date, issue_records.due_date,
issue_records.return_date,
              issue_records.status
       FROM issue_records
       INNER JOIN books ON books.id = issue_records.book_id
       WHERE issue_records.student_id = ?
       ORDER BY issue_records.created_at DESC`,
      [request.user.id]
    );

    return response.json({
      success: true,
      issues: issues.map(addOverdueFlag)
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to fetch your issued books"
    });
  }
```

```
}

async function returnIssue(request, response) {
  try {
    const issueId = Number(request.params.id);

    if (!issueId) {
      return response.status(400).json({
        success: false,
        message: "Invalid issue id"
      });
    }

    const issue = await get("SELECT * FROM issue_records WHERE id = ?",
[issueId]);

    if (!issue) {
      return response.status(404).json({
        success: false,
        message: "Issue record not found"
      });
    }

    if (issue.status === "returned") {
      return response.status(400).json({
        success: false,
        message: "Book is already returned"
      });
    }

    const returnDate = new Date().toISOString().split("T")[0];

    await run(
      `UPDATE issue_records
       SET status = 'returned', return_date = ?, updated_at =
CURRENT_TIMESTAMP
       WHERE id = ?`,
      [returnDate, issueId]
    );

    await run(
      `UPDATE books
       SET available_copies = available_copies + 1, updated_at =
CURRENT_TIMESTAMP
       WHERE id = ?`,
      [issue.book_id]
    );

    return response.json({
```

```
      success: true,
      message: "Book returned successfully"
    });
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to return book"
    });
  }
}

async function getDashboardSummary(request, response) {
  try {
    const totalBooksRow = await get("SELECT COUNT(*) AS count FROM books");
    const totalStudentsRow = await get(
      "SELECT COUNT(*) AS count FROM users WHERE role = 'student'"
    );
    const totalIssuedBooksRow = await get(
      "SELECT COUNT(*) AS count FROM issue_records WHERE status = 'issued'"
    );
    const totalAvailableBooksRow = await get(
      "SELECT COALESCE(SUM(available_copies), 0) AS count FROM books"
    );
    const recentBooks = await all(
      "SELECT id, title, author, category, created_at FROM books ORDER BY
created_at DESC LIMIT 5"
    );
    const recentIssues = await all(
      `SELECT issue_records.id, users.full_name AS student_name, books.title
AS book_title,
              issue_records.issue_date, issue_records.due_date,
issue_records.status
       FROM issue_records
       INNER JOIN users ON users.id = issue_records.student_id
       INNER JOIN books ON books.id = issue_records.book_id
       ORDER BY issue_records.created_at DESC
       LIMIT 5`
    );

    return response.json({
      success: true,
      summary: {
        total_books: totalBooksRow.count,
        total_students: totalStudentsRow.count,
        total_issued_books: totalIssuedBooksRow.count,
        total_available_books: totalAvailableBooksRow.count
      },
      recentBooks,
      recentIssues: recentIssues.map(addOverdueFlag)
    });
```

```
  } catch (error) {
    return response.status(500).json({
      success: false,
      message: "Failed to fetch dashboard summary"
    });
  }
}

module.exports = {
  createIssue,
  getIssues,
  getMyIssues,
  returnIssue,
  getDashboardSummary
};
```

File: backend/routes/authRoutes.js

Description: Defines authentication-related API endpoints and connects them with controller methods.

Code:

```
const express = require("express");
const authMiddleware = require("../middleware/authMiddleware");
const {
  registerStudent,
  loginUser,
  getCurrentUser
} = require("../controllers/authController");

const router = express.Router();

router.post("/register", registerStudent);
router.post("/login", loginUser);
router.get("/me", authMiddleware, getCurrentUser);

module.exports = router;
```

File: backend/routes/bookRoutes.js

Description: Defines book-related API endpoints for admin and student access.

Code:

```
const express = require("express");
const authMiddleware = require("../middleware/authMiddleware");
const roleMiddleware = require("../middleware/roleMiddleware");
const {
  getBooks,
```

```
  getBookById,
  addBook,
  updateBook,
  deleteBook
} = require("../controllers/bookController");

const router = express.Router();

router.use(authMiddleware);
router.get("/", getBooks);
router.get("/:id", getBookById);
router.post("/", roleMiddleware("admin"), addBook);
router.put("/:id", roleMiddleware("admin"), updateBook);
router.delete("/:id", roleMiddleware("admin"), deleteBook);

module.exports = router;
```

File: backend/routes/userRoutes.js

Description: Defines student and profile related API endpoints.

Code:

```
const express = require("express");
const authMiddleware = require("../middleware/authMiddleware");
const roleMiddleware = require("../middleware/roleMiddleware");
const {
  getStudents,
  getStudentById,
  updateProfile,
  deleteStudent
} = require("../controllers/userController");

const router = express.Router();

router.use(authMiddleware);
router.get("/students", roleMiddleware("admin"), getStudents);
router.get("/students/:id", roleMiddleware("admin"), getStudentById);
router.put("/profile", roleMiddleware("student"), updateProfile);
router.delete("/students/:id", roleMiddleware("admin"), deleteStudent);

module.exports = router;
```

File: backend/routes/issueRoutes.js

Description: Defines issue and return related API endpoints and dashboard summary routes.

Code:

```
const express = require("express");
const authMiddleware = require("../middleware/authMiddleware");
const roleMiddleware = require("../middleware/roleMiddleware");
const {
  createIssue,
  getIssues,
  getMyIssues,
  returnIssue
} = require("../controllers/issueController");

const router = express.Router();

router.use(authMiddleware);
router.post("/", roleMiddleware("admin"), createIssue);
router.get("/", roleMiddleware("admin"), getIssues);
router.get("/my", roleMiddleware("student"), getMyIssues);
router.put("/:id/return", roleMiddleware("admin"), returnIssue);

module.exports = router;
```

File: backend/database/schema.sql

Description: Contains the SQL statements used to create the main database tables and constraints.

Code:

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  full_name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password TEXT NOT NULL,
  role TEXT NOT NULL CHECK(role IN ('admin', 'student')),
  roll_number TEXT UNIQUE,
  course TEXT,
  phone_number TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS books (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  author TEXT NOT NULL,
  category TEXT NOT NULL,
  isbn TEXT NOT NULL UNIQUE,
  total_copies INTEGER NOT NULL CHECK(total_copies >= 1),
  available_copies INTEGER NOT NULL CHECK(available_copies >= 0),
```

```sql
  shelf_location TEXT,
  description TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS issue_records (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  student_id INTEGER NOT NULL,
  book_id INTEGER NOT NULL,
  issue_date TEXT NOT NULL,
  due_date TEXT NOT NULL,
  return_date TEXT,
  status TEXT NOT NULL CHECK(status IN ('issued', 'returned')),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (student_id) REFERENCES users(id),
  FOREIGN KEY (book_id) REFERENCES books(id)
);
```

File: backend/database/seed.sql

Description: Contains initial seed data used during first-time database setup.

Code:

```sql
INSERT INTO users (full_name, email, password, role)
VALUES ('Library Admin', 'admin@library.com', '{{ADMIN_PASSWORD_HASH}}',
'admin');
```

File: public/add-book.html

Description: Form page used by the admin to add a new book.

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Add Book</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/form.css">
</head>
<body data-page="add-book">
  <div id="navbar"></div>
  <main class="page-container narrow-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">New Entry</span>
```

```
        <h1>Add Book</h1>
        <p class="page-subtitle">Add a new book to the college library with
copies, location, and description details.</p>
      </div>
    </section>
    <section class="card content-card">
      <form id="bookForm" class="form-grid">
        <label>Title
          <input type="text" name="title" required>
        </label>
        <label>Author
          <input type="text" name="author" required>
        </label>
        <label>Category
          <input type="text" name="category" required>
        </label>
        <label>ISBN
          <input type="text" name="isbn" required>
        </label>
        <label>Total Copies
          <input type="number" name="total_copies" min="1" required>
        </label>
        <label>Shelf Location
          <input type="text" name="shelf_location">
        </label>
        <label class="full-width">Description
          <textarea name="description" rows="4"></textarea>
        </label>
        <button type="submit" class="btn btn-primary">Save Book</button>
      </form>
      <div id="messageBox"></div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/add-book.js"></script>
</body>
</html>
```

File: public/admin-books.html

Description: Admin page for viewing and managing all books.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Manage Books</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/table.css">
  <link rel="stylesheet" href="css/books.css">
</head>
<body data-page="admin-books">
  <div id="navbar"></div>
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">Book Management</span>
        <h1>Books</h1>
        <p class="page-subtitle">View, search, add, edit, and manage the
complete library book collection with a consistent college theme.</p>
      </div>
      <div class="page-hero-side">
        <div class="highlight-box">
          <strong>Books</strong>
          <span>Track availability and copies</span>
        </div>
      </div>
    </section>
    <section class="page-header">
      <h1>Books</h1>
      <a class="btn btn-primary" href="add-book.html">Add Book</a>
    </section>

    <section class="card filter-card content-card">
      <form id="bookSearchForm" class="inline-form">
        <input type="text" name="search" placeholder="Search by title or
author">
        <input type="text" name="category" placeholder="Filter by category">
        <button type="submit" class="btn btn-primary">Search</button>
        <button type="button" id="resetFilters" class="btn
btn-secondary">Reset</button>
      </form>
    </section>

    <div id="messageBox"></div>
    <section class="card content-card">
      <div class="table-wrapper">
        <table>
          <thead>
            <tr>
              <th>Title</th>
              <th>Author</th>
              <th>Category</th>
              <th>ISBN</th>
              <th>Total</th>
```

```
              <th>Available</th>
              <th>Status</th>
              <th>Actions</th>
            </tr>
          </thead>
          <tbody id="booksTableBody"></tbody>
        </table>
      </div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/admin-books.js"></script>
</body>
</html>
```

File: public/admin-dashboard.html

Description: Admin dashboard page showing library summary information.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin Dashboard</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/dashboard.css">
</head>
<body data-page="admin-dashboard">
  <div id="navbar"></div>
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">Admin Panel</span>
        <h1>Admin Dashboard</h1>
        <p class="page-subtitle">Manage books, students, and issue records
for Post Graduate Government College for Girls in one place.</p>
      </div>
      <div class="page-hero-side">
        <div class="highlight-box">
          <strong>Library</strong>
          <span>College Management Portal</span>
        </div>
      </div>
    </section>
    <div id="dashboardMessage"></div>
```

```
    <section id="summaryCards" class="grid-4"></section>
    <section class="grid-2">
      <div class="card content-card">
        <h2>Recently Added Books</h2>
        <div id="recentBooksList"></div>
      </div>
      <div class="card content-card">
        <h2>Recently Issued Records</h2>
        <div id="recentIssuesList"></div>
      </div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/admin-dashboard.js"></script>
</body>
</html>
```

File: public/book-details.html

Description: Displays detailed information of a selected book.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Book Details</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/books.css">
</head>
<body data-page="book-details">
  <div id="navbar"></div>
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">Book Information</span>
        <h1>Book Details</h1>
        <p class="page-subtitle">View complete information about the selected
book, including location and availability.</p>
      </div>
    </section>
    <div id="messageBox"></div>
    <section class="card content-card" id="bookDetailsCard"></section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
```

```
  <script src="js/common.js"></script>
  <script src="js/book-details.js"></script>
</body>
</html>
```

File: public/edit-book.html

Description: Form page used by the admin to edit an existing book.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Edit Book</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/form.css">
</head>
<body data-page="edit-book">
  <div id="navbar"></div>
  <main class="page-container narrow-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">Update Record</span>
        <h1>Edit Book</h1>
        <p class="page-subtitle">Update book information while keeping the
total and available copies logic correct.</p>
      </div>
    </section>
    <section class="card content-card">
      <form id="editBookForm" class="form-grid">
        <label>Title
          <input type="text" name="title" required>
        </label>
        <label>Author
          <input type="text" name="author" required>
        </label>
        <label>Category
          <input type="text" name="category" required>
        </label>
        <label>Total Copies
          <input type="number" name="total_copies" min="1" required>
        </label>
        <label>Shelf Location
          <input type="text" name="shelf_location">
        </label>
        <label class="full-width">Description
          <textarea name="description" rows="4"></textarea>
```

```
        </label>
        <button type="submit" class="btn btn-primary">Update Book</button>
      </form>
      <div id="messageBox"></div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/edit-book.js"></script>
</body>
</html>
```

File: public/index.html

Description: Home page of the project that introduces the system and provides navigation links.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Campus Library Management System</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body data-page="home">
  <div id="navbar"></div>
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">College Library System</span>
        <h1>Campus Library Management System</h1>
        <p class="page-subtitle">This project helps the library of Post
Graduate Government College for Girls manage books, students, and issue
records in a simple and clear way.</p>
        <p><strong>College:</strong> Post Graduate Government College for
Girls</p>
        <p><strong>Address:</strong> Sector-11, Chandigarh</p>
        <div class="hero-actions">
          <a class="btn btn-primary" href="login.html">Login</a>
          <a class="btn btn-secondary" href="register.html">Student
Register</a>
        </div>
      </div>
      <div class="page-hero-side">
        <div class="highlight-box">
          <strong>PGGCG</strong>
```

```
          <span>Simple and neat library management</span>
        </div>
      </div>
    </section>

    <section class="grid-3">
      <article class="card content-card">
        <h2>Admin Features</h2>
        <p>Manage books, students, issue records, and return books.</p>
      </article>
      <article class="card content-card">
        <h2>Student Features</h2>
        <p>View books, search by title or author, and check your issued
books.</p>
      </article>
      <article class="card content-card">
        <h2>Simple UI</h2>
        <p>Built with plain HTML, CSS, JavaScript, Node.js, Express, and
SQLite.</p>
      </article>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/common.js"></script>
</body>
</html>
```

File: public/issue-book.html

Description: Admin page used to issue a book to a student.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Issue Book</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/form.css">
  <link rel="stylesheet" href="css/issue.css">
</head>
<body data-page="issue-book">
  <div id="navbar"></div>
  <main class="page-container narrow-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">Issue Management</span>
        <h1>Issue Book</h1>
```

```
        <p class="page-subtitle">Select a student, choose an available book,
and set the due date for issue.</p>
      </div>
    </section>
    <section class="card content-card">
      <p class="issue-note">Only books with available copies can be issued.
Duplicate active issues are blocked automatically.</p>
      <form id="issueForm" class="form-grid">
        <label>Student
          <select name="student_id" id="studentSelect" required></select>
        </label>
        <label>Book
          <select name="book_id" id="bookSelect" required></select>
        </label>
        <label>Due Date
          <input type="date" name="due_date" id="dueDateInput" required>
        </label>
        <button type="submit" class="btn btn-primary">Issue Book</button>
      </form>
      <div id="messageBox"></div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/issue-book.js"></script>
</body>
</html>
```

File: public/issued-records.html

Description: Displays all issue and return records for the admin.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Issued Records</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/table.css">
  <link rel="stylesheet" href="css/issue.css">
</head>
<body data-page="issued-records">
  <div id="navbar"></div>
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
```

```
        <span class="page-kicker">Issue History</span>
        <h1>Issued Records</h1>
        <p class="page-subtitle">Monitor all issue and return transactions,
including overdue books, from a single screen.</p>
      </div>
      <div class="page-hero-side">
        <div class="highlight-box">
          <strong>Records</strong>
          <span>Issue and return tracking</span>
        </div>
      </div>
    </section>
    <div id="messageBox"></div>
    <section class="card content-card">
      <div class="table-wrapper">
        <table>
          <thead>
            <tr>
              <th>Student</th>
              <th>Book</th>
              <th>Issue Date</th>
              <th>Due Date</th>
              <th>Return Date</th>
              <th>Status</th>
              <th>Overdue</th>
              <th>Action</th>
            </tr>
          </thead>
          <tbody id="issueRecordsTableBody"></tbody>
        </table>
      </div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/issued-records.js"></script>
</body>
</html>
```

File: public/login.html

Description: Login page for admin and student users.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/auth.css">
</head>
<body data-page="login">
  <div id="navbar"></div>
  <main class="page-container narrow-container">
    <section class="card auth-card content-card">
      <div class="auth-header">
        <span class="page-kicker">Secure Access</span>
        <h1>Login</h1>
        <p>Use your admin or student account to continue in the college
library portal.</p>
      </div>
      <form id="loginForm" class="form-grid">
        <label>Email
          <input type="email" name="email" required>
        </label>
        <label>Password
          <input type="password" name="password" required>
        </label>
        <button type="submit" class="btn btn-primary">Login</button>
      </form>
      <p class="form-note">Student account? <a href="register.html">Register
here</a></p>
      <div id="messageBox"></div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/auth.js"></script>
</body>
</html>
```

File: public/my-issued-books.html

Description: Shows the books currently or previously issued to the logged-in student.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Issued Books</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/table.css">
```

```html
  <link rel="stylesheet" href="css/issue.css">
</head>
<body data-page="my-issued-books">
  <div id="navbar"></div>
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">My Borrowing History</span>
        <h1>My Issued Books</h1>
        <p class="page-subtitle">See your current and past issued books along
with due dates, return dates, and overdue status.</p>
      </div>
    </section>
    <div id="messageBox"></div>
    <section class="card content-card">
      <div class="table-wrapper">
        <table>
          <thead>
            <tr>
              <th>Book</th>
              <th>Author</th>
              <th>Category</th>
              <th>Issue Date</th>
              <th>Due Date</th>
              <th>Return Date</th>
              <th>Status</th>
              <th>Overdue</th>
            </tr>
          </thead>
          <tbody id="myIssuesTableBody"></tbody>
        </table>
      </div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/my-issued-books.js"></script>
</body>
</html>
```

File: public/not-found.html

Description: Error page shown when a page or route is not found.

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
```

```
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Not Found</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body data-page="not-found">
  <div id="navbar"></div>
  <main class="page-container narrow-container">
    <section class="card content-card">
      <span class="page-kicker">Error</span>
      <h1>Page Not Found</h1>
      <p>The page you are looking for does not exist.</p>
      <a class="btn btn-primary" href="index.html">Back to Home</a>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/common.js"></script>
</body>
</html>
```

File: public/profile.html

Description: Profile page for viewing and updating user information.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Profile</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/form.css">
  <link rel="stylesheet" href="css/profile.css">
</head>
<body data-page="profile">
  <div id="navbar"></div>
  <main class="page-container narrow-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">Account Details</span>
        <h1>My Profile</h1>
        <p class="page-subtitle">View your account information and update
basic details if you are logged in as a student.</p>
      </div>
    </section>
    <section class="card content-card">
      <form id="profileForm" class="form-grid">
        <label>Full Name
```

```
          <input type="text" name="full_name" required>
        </label>
        <label>Email
          <input type="email" name="email" disabled>
        </label>
        <label>Roll Number
          <input type="text" name="roll_number" disabled>
        </label>
        <label>Course
          <input type="text" name="course" required>
        </label>
        <label>Phone Number
          <input type="text" name="phone_number" required>
        </label>
        <button id="profileSubmitButton" type="submit" class="btn
btn-primary">Update Profile</button>
      </form>
      <div id="messageBox"></div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/profile.js"></script>
</body>
</html>
```

File: public/register.html

Description: Student registration page.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Register</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/auth.css">
  <link rel="stylesheet" href="css/form.css">
</head>
<body data-page="register">
  <div id="navbar"></div>
  <main class="page-container narrow-container">
    <section class="card content-card">
      <div class="auth-header">
        <span class="page-kicker">New Student</span>
        <h1>Student Registration</h1>
```

```html
        <p>Create your account to browse books and track issued records.</p>
      </div>
      <form id="registerForm" class="form-grid">
        <label>Full Name
          <input type="text" name="full_name" required>
        </label>
        <label>Email
          <input type="email" name="email" required>
        </label>
        <label>Password
          <input type="password" name="password" minlength="6" required>
        </label>
        <label>Roll Number
          <input type="text" name="roll_number" required>
        </label>
        <label>Course
          <input type="text" name="course" required>
        </label>
        <label>Phone Number
          <input type="text" name="phone_number" required>
        </label>
        <button type="submit" class="btn btn-primary">Register</button>
      </form>
      <div id="messageBox"></div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/auth.js"></script>
</body>
</html>
```

File: public/student-books.html

Description: Displays book listing for student users.

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Browse Books</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/books.css">
</head>
<body data-page="student-books">
  <div id="navbar"></div>
```

```
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">Book Catalogue</span>
        <h1>Browse Books</h1>
        <p class="page-subtitle">Search by title or author, filter by
category, and view book details before visiting the library.</p>
      </div>
      <div class="page-hero-side">
        <div class="highlight-box">
          <strong>Explore</strong>
          <span>Find available books quickly</span>
        </div>
      </div>
    </section>
    <section class="card filter-card content-card">
      <form id="studentBookSearchForm" class="inline-form">
        <input type="text" name="search" placeholder="Search by title or
author">
        <input type="text" name="category" placeholder="Filter by category">
        <button type="submit" class="btn btn-primary">Search</button>
        <button type="button" id="resetBookFilters" class="btn
btn-secondary">Reset</button>
      </form>
    </section>
    <div id="messageBox"></div>
    <section id="studentBooksGrid" class="grid-3"></section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/student-books.js"></script>
</body>
</html>
```

File: public/student-dashboard.html

Description: Student dashboard page for student-specific actions and overview.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Dashboard</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/dashboard.css">
</head>
```

```html
<body data-page="student-dashboard">
  <div id="navbar"></div>
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">Student Panel</span>
        <h1 id="studentWelcome">Student Dashboard</h1>
        <p class="page-subtitle">Check your current issued books, due dates,
and overdue status from one simple dashboard.</p>
      </div>
      <div class="page-hero-side">
        <div class="highlight-box">
          <strong>My Library</strong>
          <span>Books, due dates, and profile</span>
        </div>
      </div>
    </section>
    <section id="studentSummary" class="grid-4"></section>
    <section class="card content-card">
      <h2>My Current Issued Books</h2>
      <div id="studentCurrentIssues"></div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/student-dashboard.js"></script>
</body>
</html>
```

File: public/student-details.html

Description: Displays complete details of a selected student and related issue records.

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Details</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/table.css">
</head>
<body data-page="student-details">
  <div id="navbar"></div>
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
```

```
        <span class="page-kicker">Student Profile</span>
        <h1>Student Details</h1>
        <p class="page-subtitle">Review profile information, active books,
and return history in a clear format.</p>
      </div>
    </section>
    <div id="messageBox"></div>
    <section class="card content-card" id="studentInfo"></section>
    <section class="card content-card">
      <h2>Issued Books</h2>
      <div class="table-wrapper">
        <table>
          <thead>
            <tr>
              <th>Book</th>
              <th>Issue Date</th>
              <th>Due Date</th>
              <th>Return Date</th>
              <th>Status</th>
            </tr>
          </thead>
          <tbody id="studentIssueTableBody"></tbody>
        </table>
      </div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/student-details.js"></script>
</body>
</html>
```

File: public/students.html

Description: Displays the student list for admin management.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Students</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/table.css">
</head>
<body data-page="students">
  <div id="navbar"></div>
```

```
  <main class="page-container">
    <section class="hero-card page-hero">
      <div class="page-hero-content">
        <span class="page-kicker">Student Records</span>
        <h1>Students</h1>
        <p class="page-subtitle">View all registered students, their basic
information, and access complete borrowing details.</p>
      </div>
      <div class="page-hero-side">
        <div class="highlight-box">
          <strong>Students</strong>
          <span>Simple record management</span>
        </div>
      </div>
    </section>
    <div id="messageBox"></div>
    <section class="card content-card">
      <div class="table-wrapper">
        <table>
          <thead>
            <tr>
              <th>Name</th>
              <th>Email</th>
              <th>Roll Number</th>
              <th>Course</th>
              <th>Phone</th>
              <th>Status</th>
              <th>Details</th>
            </tr>
          </thead>
          <tbody id="studentsTableBody"></tbody>
        </table>
      </div>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/api.js"></script>
  <script src="js/common.js"></script>
  <script src="js/students.js"></script>
</body>
</html>
```

File: public/unauthorized.html

Description: Error page shown when a user tries to access a restricted page.

Code:

```
<!DOCTYPE html>
<html lang="en">
```

```html
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Unauthorized</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body data-page="unauthorized">
  <div id="navbar"></div>
  <main class="page-container narrow-container">
    <section class="card content-card">
      <span class="page-kicker">Access Denied</span>
      <h1>Unauthorized Access</h1>
      <p>You do not have permission to access this page.</p>
      <a class="btn btn-primary" href="login.html">Go to Login</a>
    </section>
  </main>
  <div id="footer"></div>
  <script src="js/common.js"></script>
</body>
</html>
```

File: public/js/add-book.js

Description: Handles frontend logic for adding a new book.

Code:

```javascript
document.addEventListener("DOMContentLoaded", () => {
  const user = requireAuth("admin");

  if (!user) {
    return;
  }

  document.getElementById("bookForm").addEventListener("submit", async
(event) => {
    event.preventDefault();
    const formData = new FormData(event.target);
    const payload = Object.fromEntries(formData.entries());

    if (Number(payload.total_copies) < 1) {
      showMessage("messageBox", "Total copies must be a positive number",
"error");
      return;
    }

    try {
      const result = await apiRequest("/books", {
        method: "POST",
        body: JSON.stringify(payload)
```

```
      });

      showMessage("messageBox", result.message, "success");
      event.target.reset();
    } catch (error) {
      showMessage("messageBox", error.message, "error");
    }
  });
});
```

File: public/js/admin-books.js

Description: Handles book listing, search, filter, and delete actions on the admin books page.

Code:

```
async function loadAdminBooks(search = "", category = "") {
  try {
    const query = new URLSearchParams();

    if (search) {
      query.append("search", search);
    }

    if (category) {
      query.append("category", category);
    }

    const result = await apiRequest(`/books?${query.toString()}`);
    const books = result.books || [];
    const tableBody = document.getElementById("booksTableBody");

    if (!books.length) {
      tableBody.innerHTML = `<tr><td colspan="8">No books found</td></tr>`;
      return;
    }

    tableBody.innerHTML = books
      .map(
        (book) => `
          <tr>
            <td>${book.title}</td>
            <td>${book.author}</td>
            <td>${book.category}</td>
            <td>${book.isbn}</td>
            <td>${book.total_copies}</td>
            <td>${book.available_copies}</td>
            <td>${book.available_copies > 0 ? createBadge("Available",
"available") : createBadge("Not Available", "unavailable")}</td>
```

```
            <td>
              <a class="btn btn-secondary"
href="edit-book.html?id=${book.id}">Edit</a>
              <button class="btn btn-danger"
onclick="deleteBook(${book.id})">Delete</button>
            </td>
          </tr>
        `
      )
      .join("");
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }
}

async function deleteBook(bookId) {
  const confirmed = window.confirm("Are you sure you want to delete this
book?");

  if (!confirmed) {
    return;
  }

  try {
    const result = await apiRequest(`/books/${bookId}`, {
      method: "DELETE"
    });

    showMessage("messageBox", result.message, "success");
    loadAdminBooks();
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }
}

document.addEventListener("DOMContentLoaded", () => {
  const user = requireAuth("admin");

  if (!user) {
    return;
  }

  loadAdminBooks();

  document.getElementById("bookSearchForm").addEventListener("submit",
(event) => {
    event.preventDefault();
    const formData = new FormData(event.target);
    loadAdminBooks(formData.get("search"), formData.get("category"));
```

```
  });

  document.getElementById("resetFilters").addEventListener("click", () => {
    document.getElementById("bookSearchForm").reset();
    loadAdminBooks();
  });
});
```

File: public/js/admin-dashboard.js

Description: Handles admin dashboard summary data rendering.

Code:

```
document.addEventListener("DOMContentLoaded", async () => {
  const user = requireAuth("admin");

  if (!user) {
    return;
  }

  try {
    const [booksResult, studentsResult, issuesResult] = await Promise.all([
      apiRequest("/books"),
      apiRequest("/users/students"),
      apiRequest("/issues")
    ]);

    const books = booksResult.books || [];
    const students = studentsResult.students || [];
    const issues = issuesResult.issues || [];
    const totalAvailableBooks = books.reduce(
      (sum, book) => sum + Number(book.available_copies),
      0
    );

    document.getElementById("summaryCards").innerHTML = `
      <div class="summary-card"><h3>Total
Books</h3><p>${books.length}</p></div>
      <div class="summary-card"><h3>Total
Students</h3><p>${students.length}</p></div>
      <div class="summary-card"><h3>Total Issued
Books</h3><p>${issues.filter((issue) => issue.status ===
"issued").length}</p></div>
      <div class="summary-card"><h3>Total Available
Books</h3><p>${totalAvailableBooks}</p></div>
    `;

    const recentBooks = books.slice(0, 5);
    const recentIssues = issues.slice(0, 5);
```

```
    document.getElementById("recentBooksList").innerHTML = recentBooks.length
      ? `<ul class="styled-list">${recentBooks
          .map(
            (book) => `
              <li>
                <strong>${book.title}</strong>
                <div class="muted-text">${book.author} |
${book.category}</div>
              </li>
            `
          )
          .join("")}</ul>`
      : `<p class="empty-state">No books found</p>`;

    document.getElementById("recentIssuesList").innerHTML =
recentIssues.length
      ? `<ul class="styled-list">${recentIssues
          .map(
            (issue) => `
              <li>
                <strong>${issue.student_name}</strong>
                <div class="muted-text">${issue.book_title} | Due
${formatDate(issue.due_date)}</div>
              </li>
            `
          )
          .join("")}</ul>`
      : `<p class="empty-state">No issued books found</p>`;
  } catch (error) {
    showMessage("dashboardMessage", error.message, "error");
  }
});
```

File: public/js/api.js

Description: Contains common API request and local storage helper functions.

Code:

```
const API_BASE_URL = "/api";

function getToken() {
  return localStorage.getItem("token");
}

function getCurrentUser() {
  const user = localStorage.getItem("user");
  return user ? JSON.parse(user) : null;
}
```

```
function saveAuthData(token, user) {
  localStorage.setItem("token", token);
  localStorage.setItem("user", JSON.stringify(user));
}

function clearAuthData() {
  localStorage.removeItem("token");
  localStorage.removeItem("user");
}

async function apiRequest(endpoint, options = {}) {
  const headers = {
    "Content-Type": "application/json",
    ...(options.headers || {})
  };

  if (getToken()) {
    headers.Authorization = `Bearer ${getToken()}`;
  }

  const response = await fetch(`${API_BASE_URL}${endpoint}`, {
    ...options,
    headers
  });

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || "Request failed");
  }

  return data;
}
```

File: public/js/auth.js

Description: Handles login and registration form submission on the frontend.

Code:

```
document.addEventListener("DOMContentLoaded", () => {
  const pageName = document.body.dataset.page;
  const currentUser = getCurrentUser();

  if ((pageName === "login" || pageName === "register") && currentUser) {
    redirectToDashboardByRole(currentUser.role);
    return;
  }
```

```
const loginForm = document.getElementById("loginForm");
const registerForm = document.getElementById("registerForm");

if (loginForm) {
  loginForm.addEventListener("submit", async (event) => {
    event.preventDefault();
    showMessage("messageBox", "", "success");

    const formData = new FormData(loginForm);
    const payload = Object.fromEntries(formData.entries());

    try {
      const result = await apiRequest("/auth/login", {
        method: "POST",
        body: JSON.stringify(payload)
      });

      saveAuthData(result.token, result.user);
      redirectToDashboardByRole(result.user.role);
    } catch (error) {
      showMessage("messageBox", error.message, "error");
    }
  });
}

if (registerForm) {
  registerForm.addEventListener("submit", async (event) => {
    event.preventDefault();
    showMessage("messageBox", "", "success");

    const formData = new FormData(registerForm);
    const payload = Object.fromEntries(formData.entries());

    if (payload.password.length < 6) {
      showMessage("messageBox", "Password must be at least 6 characters",
"error");
      return;
    }

    try {
      const result = await apiRequest("/auth/register", {
        method: "POST",
        body: JSON.stringify(payload)
      });

      showMessage("messageBox", result.message, "success");
      registerForm.reset();
      setTimeout(() => {
```

```
          window.location.href = "login.html";
        }, 1200);
      } catch (error) {
        showMessage("messageBox", error.message, "error");
      }
    });
  }
});
```

File: public/js/book-details.js

Description: Loads and displays a selected book on the details page.

Code:

```
document.addEventListener("DOMContentLoaded", async () => {
  const user = requireAuth("student");

  if (!user) {
    return;
  }

  const bookId = getQueryParam("id");

  if (!bookId) {
    showMessage("messageBox", "Invalid book id", "error");
    return;
  }

  try {
    const result = await apiRequest(`/books/${bookId}`);
    const book = result.book;

    document.getElementById("bookDetailsCard").innerHTML = `
      <h1>${book.title}</h1>
      <p class="section-subtitle">Complete information about the selected
library book</p>
      <div class="detail-grid">
        <div
class="detail-item"><span>Author</span><strong>${book.author}</strong></div>
        <div
class="detail-item"><span>Category</span><strong>${book.category}</strong></d
iv>
        <div
class="detail-item"><span>ISBN</span><strong>${book.isbn}</strong></div>
        <div class="detail-item"><span>Shelf
Location</span><strong>${book.shelf_location || "-"}</strong></div>
      </div>
      <div class="inner-panel">
        <h2>Description</h2>
```

```
        <p>${book.description || "No description available"}</p>
        <p><strong>Availability:</strong> ${book.available_copies > 0 ?
createBadge("Available", "available") : createBadge("Not Available",
"unavailable")}</p>
      </div>
    `;
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }
});
```

File: public/js/common.js

Description: Contains shared frontend utilities such as navbar rendering, footer rendering, date formatting, and authentication checks.

Code:

```
function getPageName() {
  return document.body.dataset.page;
}

function isActivePage(pageNames) {
  return pageNames.includes(getPageName());
}

function showMessage(elementId, message, type) {
  const element = document.getElementById(elementId);

  if (!element) {
    return;
  }

  if (!message) {
    element.innerHTML = "";
    return;
  }

  element.innerHTML = `<div class="message
message-${type}">${message}</div>`;
}

function createBadge(text, type) {
  return `<span class="badge badge-${type}">${text}</span>`;
}

function formatDate(dateValue) {
  if (!dateValue) {
    return "-";
```

```
  }

  return new Date(dateValue).toLocaleDateString("en-IN");
}

function getQueryParam(name) {
  const searchParams = new URLSearchParams(window.location.search);
  return searchParams.get(name);
}

function redirectToDashboardByRole(role) {
  if (role === "admin") {
    window.location.href = "admin-dashboard.html";
    return;
  }

  window.location.href = "student-dashboard.html";
}

function requireAuth(requiredRole) {
  const user = typeof getCurrentUser === "function" ? getCurrentUser() :
null;

  if (!user) {
    window.location.href = "login.html";
    return null;
  }

  if (requiredRole && user.role !== requiredRole) {
    window.location.href = "unauthorized.html";
    return null;
  }

  return user;
}

function renderNavbar() {
  const navbar = document.getElementById("navbar");

  if (!navbar) {
    return;
  }

  const user = typeof getCurrentUser === "function" ? getCurrentUser() :
null;
  const guestHomeClass = isActivePage(["home"]) ? "active-link" : "";
  const guestLoginClass = isActivePage(["login"]) ? "active-link" : "";
  const guestRegisterClass = isActivePage(["register"]) ? "active-link" : "";
  let links = `
```

```
    <a href="index.html" class="${guestHomeClass}">Home</a>
    <a href="login.html" class="${guestLoginClass}">Login</a>
    <a href="register.html" class="${guestRegisterClass}">Register</a>
  `;

  if (user && user.role === "student") {
    const dashboardClass = isActivePage(["student-dashboard"]) ?
"active-link" : "";
    const booksClass = isActivePage(["student-books", "book-details"]) ?
"active-link" : "";
    const issuesClass = isActivePage(["my-issued-books"]) ? "active-link" :
"";
    const profileClass = isActivePage(["profile"]) ? "active-link" : "";

    links = `
      <a href="student-dashboard.html"
class="${dashboardClass}">Dashboard</a>
      <a href="student-books.html" class="${booksClass}">Books</a>
      <a href="my-issued-books.html" class="${issuesClass}">My Issued
Books</a>
      <a href="profile.html" class="${profileClass}">Profile</a>
      <button type="button" id="logoutButton">Logout</button>
    `;
  }

  if (user && user.role === "admin") {
    const dashboardClass = isActivePage(["admin-dashboard"]) ? "active-link"
: "";
    const booksClass = isActivePage(["admin-books", "add-book", "edit-book"])
? "active-link" : "";
    const studentsClass = isActivePage(["students", "student-details"]) ?
"active-link" : "";
    const issueBookClass = isActivePage(["issue-book"]) ? "active-link" : "";
    const issueRecordsClass = isActivePage(["issued-records"]) ?
"active-link" : "";
    const profileClass = isActivePage(["profile"]) ? "active-link" : "";

    links = `
      <a href="admin-dashboard.html" class="${dashboardClass}">Dashboard</a>
      <a href="admin-books.html" class="${booksClass}">Books</a>
      <a href="students.html" class="${studentsClass}">Students</a>
      <a href="issue-book.html" class="${issueBookClass}">Issue Book</a>
      <a href="issued-records.html" class="${issueRecordsClass}">Issued
Records</a>
      <a href="profile.html" class="${profileClass}">Profile</a>
      <button type="button" id="logoutButton">Logout</button>
    `;
  }
```

```
  navbar.innerHTML = `
    <header class="topbar">
      <div class="topbar-inner">
        <a class="brand" href="index.html">
          <img src="assets/logo.png" alt="College Logo" class="brand-logo">
          <div class="brand-text">
            <span class="brand-title">Post Graduate Government College for
Girls</span>
            <span class="brand-subtitle">Sector-11, Chandigarh</span>
          </div>
        </a>
        <nav class="nav-links">${links}</nav>
      </div>
    </header>
  `;

  const logoutButton = document.getElementById("logoutButton");

  if (logoutButton) {
    logoutButton.addEventListener("click", () => {
      clearAuthData();
      window.location.href = "login.html";
    });
  }
}

function renderFooter() {
  const footer = document.getElementById("footer");

  if (!footer) {
    return;
  }

  footer.innerHTML = `
    <footer class="footer">
      <div class="footer-content">
        <p>Campus Library Management System 2026</p>
        <p>Post Graduate Government College for Girls, Sector-11,
Chandigarh</p>
        <p>Project by Sakshi Goyal</p>
        <p>Roll No.: 1076/23, BCA - 6th Semester</p>
      </div>
    </footer>
  `;
}

document.addEventListener("DOMContentLoaded", () => {
  renderNavbar();
```

```
  renderFooter();
});
```

File: public/js/edit-book.js

Description: Handles frontend logic for editing a selected book.

Code:

```
document.addEventListener("DOMContentLoaded", async () => {
  const user = requireAuth("admin");

  if (!user) {
    return;
  }

  const bookId = getQueryParam("id");
  const form = document.getElementById("editBookForm");

  if (!bookId) {
    showMessage("messageBox", "Invalid book id", "error");
    return;
  }

  try {
    const result = await apiRequest(`/books/${bookId}`);
    const book = result.book;

    form.title.value = book.title;
    form.author.value = book.author;
    form.category.value = book.category;
    form.total_copies.value = book.total_copies;
    form.shelf_location.value = book.shelf_location || "";
    form.description.value = book.description || "";
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }

  form.addEventListener("submit", async (event) => {
    event.preventDefault();
    const formData = new FormData(form);
    const payload = Object.fromEntries(formData.entries());

    try {
      const result = await apiRequest(`/books/${bookId}`, {
        method: "PUT",
        body: JSON.stringify(payload)
      });
```

```
      showMessage("messageBox", result.message, "success");
    } catch (error) {
      showMessage("messageBox", error.message, "error");
    }
  });
});
```

File: public/js/issue-book.js

Description: Handles frontend issue-book form logic.

Code:

```
function getDefaultDueDate() {
  const today = new Date();
  today.setDate(today.getDate() + 14);
  return today.toISOString().split("T")[0];
}

document.addEventListener("DOMContentLoaded", async () => {
  const user = requireAuth("admin");

  if (!user) {
    return;
  }

  const dueDateInput = document.getElementById("dueDateInput");
  dueDateInput.min = new Date().toISOString().split("T")[0];
  dueDateInput.value = getDefaultDueDate();

  try {
    const [studentsResult, booksResult] = await Promise.all([
      apiRequest("/users/students"),
      apiRequest("/books")
    ]);

    const students = studentsResult.students || [];
    const books = booksResult.books || [];

    document.getElementById("studentSelect").innerHTML = `
      <option value="">Select Student</option>
      ${students.map((student) => `<option
value="${student.id}">${student.full_name}
(${student.roll_number})</option>`).join("")}
    `;

    document.getElementById("bookSelect").innerHTML = `
      <option value="">Select Book</option>
      ${books
        .map(
```

```
          (book) =>
            `<option value="${book.id}" ${book.available_copies < 1 ?
"disabled" : ""}>${book.title} (${book.available_copies} available)</option>`
        )
        .join("")}
    `;
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }

  document.getElementById("issueForm").addEventListener("submit", async
(event) => {
    event.preventDefault();
    const formData = new FormData(event.target);
    const payload = Object.fromEntries(formData.entries());

    if (payload.due_date < new Date().toISOString().split("T")[0]) {
      showMessage("messageBox", "Due date should not be before today",
"error");
      return;
    }

    try {
      const result = await apiRequest("/issues", {
        method: "POST",
        body: JSON.stringify(payload)
      });

      showMessage("messageBox", result.message, "success");
      event.target.reset();
      dueDateInput.value = getDefaultDueDate();
    } catch (error) {
      showMessage("messageBox", error.message, "error");
    }
  });
});
```

File: public/js/issued-records.js

Description: Loads and manages issue records on the admin side.

Code:

```
async function loadIssueRecords() {
  try {
    const result = await apiRequest("/issues");
    const issues = result.issues || [];
    const tableBody = document.getElementById("issueRecordsTableBody");

    if (!issues.length) {
```

```
      tableBody.innerHTML = `<tr><td colspan="8">No issued books
found</td></tr>`;
      return;
    }

    tableBody.innerHTML = issues
      .map(
        (issue) => `
          <tr>
            <td>${issue.student_name}</td>
            <td>${issue.book_title}</td>
            <td>${formatDate(issue.issue_date)}</td>
            <td>${formatDate(issue.due_date)}</td>
            <td>${formatDate(issue.return_date)}</td>
            <td>${issue.status === "issued" ? createBadge("Issued", "issued")
: createBadge("Returned", "returned")}</td>
            <td>${issue.overdue ? createBadge("Overdue", "overdue") :
"-"}</td>
            <td>${issue.status === "issued" ? `<button class="btn
btn-primary" onclick="returnBook(${issue.id})">Return</button>` : "-"}</td>
          </tr>
        `
      )
      .join("");
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }
}

async function returnBook(issueId) {
  try {
    const result = await apiRequest(`/issues/${issueId}/return`, {
      method: "PUT"
    });

    showMessage("messageBox", result.message, "success");
    loadIssueRecords();
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }
}

document.addEventListener("DOMContentLoaded", () => {
  const user = requireAuth("admin");

  if (!user) {
    return;
  }
```

```
  loadIssueRecords();
});
```

File: public/js/my-issued-books.js

Description: Loads issued-book records for the logged-in student.

Code:

```
document.addEventListener("DOMContentLoaded", async () => {
  const user = requireAuth("student");

  if (!user) {
    return;
  }

  try {
    const result = await apiRequest("/issues/my");
    const issues = result.issues || [];
    const tableBody = document.getElementById("myIssuesTableBody");

    if (!issues.length) {
      tableBody.innerHTML = `<tr><td colspan="8">No issued books
found</td></tr>`;
      return;
    }

    tableBody.innerHTML = issues
      .map(
        (issue) => `
          <tr>
            <td>${issue.book_title}</td>
            <td>${issue.author}</td>
            <td>${issue.category}</td>
            <td>${formatDate(issue.issue_date)}</td>
            <td>${formatDate(issue.due_date)}</td>
            <td>${formatDate(issue.return_date)}</td>
            <td>${issue.status === "issued" ? createBadge("Issued", "issued")
: createBadge("Returned", "returned")}</td>
            <td>${issue.overdue ? createBadge("Overdue", "overdue") :
"-"}</td>
          </tr>
        `
      )
      .join("");
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }
});
```

File: public/js/profile.js

Description: Handles profile display and update operations on the frontend.

Code:

```
document.addEventListener("DOMContentLoaded", async () => {
  const user = requireAuth();

  if (!user) {
    return;
  }

  const form = document.getElementById("profileForm");
  const submitButton = document.getElementById("profileSubmitButton");

  try {
    const result = await apiRequest("/auth/me");
    const profile = result.user;

    form.full_name.value = profile.full_name || "";
    form.email.value = profile.email || "";
    form.roll_number.value = profile.roll_number || "";
    form.course.value = profile.course || "";
    form.phone_number.value = profile.phone_number || "";

    if (profile.role === "admin") {
      form.course.disabled = true;
      form.phone_number.disabled = true;
      submitButton.style.display = "none";
    }
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }

  form.addEventListener("submit", async (event) => {
    event.preventDefault();

    if (user.role !== "student") {
      return;
    }

    const formData = new FormData(form);
    const payload = {
      full_name: formData.get("full_name"),
      course: formData.get("course"),
      phone_number: formData.get("phone_number")
    };
```

```
    try {
      const result = await apiRequest("/users/profile", {
        method: "PUT",
        body: JSON.stringify(payload)
      });

      saveAuthData(getToken(), result.user);
      showMessage("messageBox", result.message, "success");
    } catch (error) {
      showMessage("messageBox", error.message, "error");
    }
  });
});
```

File: public/js/student-books.js

Description: Handles book listing and search for student users.

Code:

```
async function loadStudentBooks(search = "", category = "") {
  try {
    const query = new URLSearchParams();

    if (search) {
      query.append("search", search);
    }

    if (category) {
      query.append("category", category);
    }

    const result = await apiRequest(`/books?${query.toString()}`);
    const books = result.books || [];
    const booksGrid = document.getElementById("studentBooksGrid");

    if (!books.length) {
      booksGrid.innerHTML = `<p class="empty-state">No books found</p>`;
      return;
    }

    booksGrid.innerHTML = books
      .map(
        (book) => `
          <article class="book-card">
            <h2>${book.title}</h2>
            <p class="book-meta">${book.author} | ${book.category}</p>
            <p>ISBN: ${book.isbn}</p>
            <p>${book.available_copies > 0 ? createBadge("Available",
"available") : createBadge("Not Available", "unavailable")}</p>
```

```
            <a class="btn btn-primary"
href="book-details.html?id=${book.id}">View Details</a>
          </article>
        `
      )
      .join("");
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }
}

document.addEventListener("DOMContentLoaded", () => {
  const user = requireAuth("student");

  if (!user) {
    return;
  }

  loadStudentBooks();

  document.getElementById("studentBookSearchForm").addEventListener("submit",
(event) => {
    event.preventDefault();
    const formData = new FormData(event.target);
    loadStudentBooks(formData.get("search"), formData.get("category"));
  });

  document.getElementById("resetBookFilters").addEventListener("click", () =>
{
    document.getElementById("studentBookSearchForm").reset();
    loadStudentBooks();
  });
});
```

File: public/js/student-dashboard.js

Description: Loads dashboard content for students.

Code:

```
document.addEventListener("DOMContentLoaded", async () => {
  const user = requireAuth("student");

  if (!user) {
    return;
  }

  document.getElementById("studentWelcome").textContent = `Welcome,
${user.full_name}`;
```

```
  try {
    const result = await apiRequest("/issues/my");
    const issues = result.issues || [];
    const activeIssues = issues.filter((issue) => issue.status === "issued");
    const overdueIssues = activeIssues.filter((issue) => issue.overdue);

    document.getElementById("studentSummary").innerHTML = `
      <div class="summary-card"><h3>Total Issued
Books</h3><p>${activeIssues.length}</p></div>
      <div class="summary-card"><h3>Overdue
Books</h3><p>${overdueIssues.length}</p></div>
      <div class="summary-card"><h3>Returned
Books</h3><p>${issues.filter((issue) => issue.status ===
"returned").length}</p></div>
      <div class="summary-card"><h3>My Role</h3><p>Student</p></div>
    `;

    document.getElementById("studentCurrentIssues").innerHTML =
activeIssues.length
      ? `<ul class="styled-list">${activeIssues
          .map(
            (issue) =>
              `<li>
                <strong>${issue.book_title}</strong>
                <div class="muted-text">Due: ${formatDate(issue.due_date)}
${issue.overdue ? "| Overdue" : "| On time"}</div>
              </li>`
          )
          .join("")}</ul>`
      : `<p class="empty-state">No issued books found</p>`;
  } catch (error) {
    document.getElementById("studentCurrentIssues").innerHTML = `<p
class="empty-state">${error.message}</p>`;
  }
});
```

File: public/js/student-details.js

Description: Loads and displays selected student details for the admin.

Code:

```
document.addEventListener("DOMContentLoaded", async () => {
  const user = requireAuth("admin");

  if (!user) {
    return;
  }
```

```
  const studentId = getQueryParam("id");

  if (!studentId) {
    showMessage("messageBox", "Invalid student id", "error");
    return;
  }

  try {
    const result = await apiRequest(`/users/students/${studentId}`);
    const student = result.student;
    const issueRecords = result.issueRecords || [];

    document.getElementById("studentInfo").innerHTML = `
      <h1>${student.full_name}</h1>
      <p class="section-subtitle">Basic student profile and borrowing
information</p>
      <div class="detail-grid">
        <div
class="detail-item"><span>Email</span><strong>${student.email}</strong></div>
        <div class="detail-item"><span>Roll
Number</span><strong>${student.roll_number}</strong></div>
        <div
class="detail-item"><span>Course</span><strong>${student.course}</strong></di
v>
        <div class="detail-item"><span>Phone
Number</span><strong>${student.phone_number}</strong></div>
      </div>
    `;

    document.getElementById("studentIssueTableBody").innerHTML =
issueRecords.length
      ? issueRecords
          .map(
            (record) => `
              <tr>
                <td>${record.title}</td>
                <td>${formatDate(record.issue_date)}</td>
                <td>${formatDate(record.due_date)}</td>
                <td>${formatDate(record.return_date)}</td>
                <td>${record.status === "issued" ? createBadge("Issued",
"issued") : createBadge("Returned", "returned")}</td>
              </tr>
            `
          )
          .join("")
      : `<tr><td colspan="5">No issued books found</td></tr>`;
  } catch (error) {
    showMessage("messageBox", error.message, "error");
```

```
  }
});
```

File: public/js/students.js

Description: Loads and manages student records on the admin side.

Code:

```
document.addEventListener("DOMContentLoaded", async () => {
  const user = requireAuth("admin");

  if (!user) {
    return;
  }

  try {
    const result = await apiRequest("/users/students");
    const students = result.students || [];
    const tableBody = document.getElementById("studentsTableBody");

    if (!students.length) {
      tableBody.innerHTML = `<tr><td colspan="7">No students
found</td></tr>`;
      return;
    }

    tableBody.innerHTML = students
      .map(
        (student) => `
          <tr>
            <td>${student.full_name}</td>
            <td>${student.email}</td>
            <td>${student.roll_number}</td>
            <td>${student.course}</td>
            <td>${student.phone_number}</td>
            <td>${student.status}</td>
            <td><a class="btn btn-secondary"
href="student-details.html?id=${student.id}">View</a></td>
          </tr>
        `
      )
      .join("");
  } catch (error) {
    showMessage("messageBox", error.message, "error");
  }
});
```

File: public/css/auth.css

Description: Styles authentication-related pages such as login and registration.

Code:

```
.auth-card {
  margin-top: 40px;
  background: linear-gradient(135deg, #ffffff 0%, #f8fbff 100%);
}

.form-note {
  margin-top: 16px;
  color: #64748b;
}

.auth-header {
  margin-bottom: 18px;
}

.auth-header p {
  margin: 8px 0 0;
}
```

File: public/css/books.css

Description: Styles book listing and book management pages.

Code:

```
.filter-card {
  padding: 18px;
  background: linear-gradient(135deg, #ffffff 0%, #f5f9ff 100%);
}

.book-card {
  background: linear-gradient(135deg, #ffffff 0%, #f8fbff 100%);
  border: 1px solid #dbe3ef;
  border-radius: 16px;
  padding: 20px;
  box-shadow: 0 14px 32px rgba(15, 23, 42, 0.07);
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.book-card h2 {
  margin-top: 0;
  margin-bottom: 0;
}
```

```
.book-meta {
  color: #64748b;
  margin: 0;
}

.book-card p {
  margin: 0;
}

.book-card .btn {
  margin-top: auto;
  width: fit-content;
}
```

File: public/css/dashboard.css

Description: Styles admin and student dashboard pages.

Code:

```
.summary-card {
  background: linear-gradient(135deg, #ffffff 0%, #f8fbff 100%);
  border: 1px solid #cfe0ff;
  border-radius: 16px;
  padding: 20px;
  box-shadow: 0 12px 28px rgba(37, 99, 235, 0.08);
}

.summary-card h3 {
  margin: 0 0 8px;
  font-size: 16px;
  color: #1e3a8a;
}

.summary-card p {
  margin: 0;
  font-size: 30px;
  font-weight: bold;
}
```

File: public/css/form.css

Description: Styles forms used in book management, profile update, and other input pages.

Code:

```
.form-grid {
  display: grid;
```

```
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 16px;
}

.form-grid label {
  display: flex;
  flex-direction: column;
  gap: 8px;
  font-weight: bold;
  color: #1e293b;
}

.form-grid input,
.form-grid textarea,
.form-grid select,
.inline-form input {
  width: 100%;
  padding: 12px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  background: #f8fbff;
  font: inherit;
  transition: border-color 0.2s ease, box-shadow 0.2s ease, background 0.2s
ease;
}

.form-grid input:focus,
.form-grid textarea:focus,
.form-grid select:focus,
.inline-form input:focus {
  outline: none;
  background: #ffffff;
  border-color: #60a5fa;
  box-shadow: 0 0 0 4px rgba(96, 165, 250, 0.16);
}

.full-width {
  grid-column: 1 / -1;
}

.form-grid button {
  width: fit-content;
}

.inline-form {
  gap: 14px;
}

.inline-form input {
```

```
  min-width: 220px;
}

@media (max-width: 700px) {
  .form-grid {
    grid-template-columns: 1fr;
  }
}
```

File: public/css/issue.css

Description: Styles issue and issue-record related pages.

Code:

```
.status-cell {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

.issue-note {
  margin-top: 0;
  color: #64748b;
}
```

File: public/css/profile.css

Description: Styles the profile page.

Code:

```
#profileForm input:disabled {
  background: #f8fafc;
  color: #64748b;
}

#profileSubmitButton {
  margin-top: 8px;
}
```

File: public/css/style.css

Description: Contains common global styles used across the frontend.

Code:

```
* {
  box-sizing: border-box;
}
```

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  background: linear-gradient(180deg, #edf4ff 0%, #f8fbff 220px, #f4f7fb
100%);
  color: #1f2937;
  line-height: 1.6;
}

a {
  color: #1d4ed8;
  text-decoration: none;
}

a:hover {
  text-decoration: underline;
}

.page-container {
  width: min(1100px, calc(100% - 32px));
  margin: 0 auto;
  flex: 1;
  padding: 24px 0 40px;
}

.narrow-container {
  width: min(760px, calc(100% - 32px));
}

.topbar {
  background: linear-gradient(135deg, #0f172a 0%, #1e3a8a 100%);
  color: #ffffff;
  padding: 14px 0;
  box-shadow: 0 8px 24px rgba(15, 23, 42, 0.16);
}

.topbar-inner {
  width: min(1100px, calc(100% - 32px));
  margin: 0 auto;
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
}

.brand {
```

```
  color: #ffffff;
  display: flex;
  align-items: center;
  gap: 12px;
}

.brand:hover {
  text-decoration: none;
}

.brand-logo {
  width: 52px;
  height: 52px;
  object-fit: contain;
  border-radius: 8px;
  background: #ffffff;
  padding: 4px;
}

.brand-text {
  display: flex;
  flex-direction: column;
  gap: 2px;
}

.brand-title {
  font-size: 18px;
  font-weight: bold;
  line-height: 1.2;
}

.brand-subtitle {
  font-size: 13px;
  color: #cbd5e1;
}

.nav-links {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}

.nav-links a,
.nav-links button {
  color: #ffffff;
  background: transparent;
  border: 1px solid rgba(255, 255, 255, 0.2);
  padding: 8px 12px;
  border-radius: 999px;
```

```
  cursor: pointer;
  font: inherit;
  transition: background 0.2s ease, border-color 0.2s ease, transform 0.2s
ease;
}

.nav-links a:hover,
.nav-links button:hover {
  text-decoration: none;
  background: rgba(255, 255, 255, 0.14);
  border-color: rgba(255, 255, 255, 0.35);
  transform: translateY(-1px);
}

.nav-links .active-link {
  background: #ffffff;
  color: #1e3a8a;
  border-color: #ffffff;
  font-weight: bold;
}

.hero-card,
.card {
  background: #ffffff;
  border: 1px solid #dbe3ef;
  border-radius: 16px;
  padding: 24px;
  margin-bottom: 20px;
  box-shadow: 0 14px 32px rgba(15, 23, 42, 0.07);
}

.hero-card {
  background: linear-gradient(135deg, #ffffff 0%, #f8fbff 100%);
  border-color: #cfe0ff;
  padding: 32px;
}

.hero-card h1,
.card h1,
.card h2 {
  margin-top: 0;
  margin-bottom: 12px;
}

.page-hero {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  gap: 20px;
```

```
  background: linear-gradient(135deg, rgba(255, 255, 255, 0.96) 0%, rgba(235,
244, 255, 0.98) 100%);
}

.page-hero-content {
  flex: 1;
}

.page-kicker {
  display: inline-block;
  margin-bottom: 10px;
  padding: 6px 12px;
  border-radius: 999px;
  background: #dbeafe;
  color: #1d4ed8;
  font-size: 12px;
  font-weight: bold;
  letter-spacing: 0.4px;
  text-transform: uppercase;
}

.page-hero h1 {
  margin: 0 0 10px;
  font-size: clamp(28px, 4vw, 40px);
  line-height: 1.2;
}

.page-subtitle {
  margin: 0;
  max-width: 760px;
  color: #475569;
}

.page-hero-side {
  min-width: 220px;
}

.highlight-box {
  padding: 18px;
  border-radius: 14px;
  background: linear-gradient(135deg, #0f172a 0%, #1e40af 100%);
  color: #ffffff;
  box-shadow: 0 14px 28px rgba(30, 64, 175, 0.2);
}

.highlight-box strong,
.highlight-box span {
  display: block;
}
```

```
.highlight-box strong {
  font-size: 28px;
  line-height: 1.1;
}

.highlight-box span {
  color: rgba(255, 255, 255, 0.82);
  font-size: 14px;
}

.hero-actions,
.page-header,
.inline-form {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  align-items: center;
}

.page-header {
  justify-content: space-between;
}

.section-title {
  margin: 0 0 6px;
  font-size: 24px;
}

.section-subtitle {
  margin: 0 0 16px;
  color: #64748b;
}

.content-card {
  position: relative;
  overflow: hidden;
}

.content-card::before {
  content: "";
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 4px;
  background: linear-gradient(90deg, #2563eb 0%, #60a5fa 100%);
}
```

```
.muted-text {
  color: #64748b;
}

.detail-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 16px;
}

.detail-item {
  padding: 14px 16px;
  border: 1px solid #dbe3ef;
  border-radius: 12px;
  background: #f8fbff;
}

.detail-item span {
  display: block;
  color: #64748b;
  font-size: 13px;
  margin-bottom: 6px;
}

.detail-item strong {
  display: block;
  color: #0f172a;
}

.action-group {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

.inner-panel {
  margin-top: 20px;
  margin-bottom: 0;
  background: #f8fbff;
  border: 1px solid #dbe3ef;
  border-radius: 14px;
  padding: 20px;
}

.btn {
  display: inline-block;
  border: none;
  border-radius: 10px;
  padding: 10px 16px;
```

```
  cursor: pointer;
  font: inherit;
  font-weight: bold;
  transition: transform 0.2s ease, box-shadow 0.2s ease, background 0.2s
ease;
}

.btn-primary {
  background: #2563eb;
  color: #ffffff;
}

.btn-primary:hover {
  background: #1d4ed8;
  text-decoration: none;
  transform: translateY(-1px);
  box-shadow: 0 10px 20px rgba(37, 99, 235, 0.2);
}

.btn-secondary {
  background: #e2e8f0;
  color: #0f172a;
}

.btn-danger {
  background: #dc2626;
  color: #ffffff;
}

.btn-secondary:hover,
.btn-danger:hover {
  text-decoration: none;
  transform: translateY(-1px);
}

.grid-2,
.grid-3,
.grid-4 {
  display: grid;
  gap: 20px;
}

.grid-2 {
  grid-template-columns: repeat(2, minmax(0, 1fr));
}

.grid-3 {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}
```

```
.grid-4 {
  grid-template-columns: repeat(4, minmax(0, 1fr));
}

.badge {
  display: inline-block;
  padding: 4px 10px;
  border-radius: 999px;
  font-size: 12px;
  font-weight: bold;
}

.badge-available {
  background: #dcfce7;
  color: #166534;
}

.badge-unavailable,
.badge-overdue {
  background: #fee2e2;
  color: #991b1b;
}

.badge-issued {
  background: #dbeafe;
  color: #1d4ed8;
}

.badge-returned {
  background: #e2e8f0;
  color: #334155;
}

.message {
  padding: 12px 14px;
  border-radius: 8px;
  margin-bottom: 16px;
  border: 1px solid transparent;
}

.message-success {
  background: #dcfce7;
  color: #166534;
  border-color: #bbf7d0;
}

.message-error {
  background: #fee2e2;
```

```
  color: #991b1b;
  border-color: #fecaca;
}

.list-box {
  margin: 0;
  padding-left: 18px;
}

.styled-list {
  list-style: none;
  margin: 0;
  padding: 0;
}

.styled-list li {
  padding: 12px 0;
  border-bottom: 1px solid #e2e8f0;
}

.styled-list li:last-child {
  border-bottom: none;
}

.empty-state {
  margin: 0;
  color: #64748b;
  padding: 16px;
  border: 1px dashed #cbd5e1;
  border-radius: 12px;
  background: #f8fbff;
}

.footer {
  background: #ffffff;
  border-top: 1px solid #dbe3ef;
  padding: 18px 0;
  margin-top: auto;
}

.footer-content {
  width: min(1100px, calc(100% - 32px));
  margin: 0 auto;
}

.footer p {
  margin: 6px 0;
  text-align: center;
  color: #64748b;
```

```
}

@media (max-width: 900px) {
  .grid-4,
  .grid-3,
  .grid-2 {
    grid-template-columns: 1fr;
  }

  .topbar-inner,
  .page-header {
    align-items: flex-start;
    flex-direction: column;
  }

  .nav-links {
    width: 100%;
  }

  .brand {
    align-items: flex-start;
  }

  .brand-logo {
    width: 44px;
    height: 44px;
  }

  .brand-title {
    font-size: 16px;
  }

  .page-hero {
    flex-direction: column;
  }

  .page-hero-side {
    width: 100%;
    min-width: 0;
  }

  .detail-grid {
    grid-template-columns: 1fr;
  }
}
```

File: public/css/table.css

Description: Styles tables used in different pages.

Code:

```
.table-wrapper {
  overflow-x: auto;
  border: 1px solid #dbe3ef;
  border-radius: 14px;
}

table {
  width: 100%;
  border-collapse: collapse;
  background: #ffffff;
}

th,
td {
  border-bottom: 1px solid #e2e8f0;
  padding: 12px;
  text-align: left;
  vertical-align: top;
}

th {
  background: #eff6ff;
  color: #1e3a8a;
  font-size: 13px;
  text-transform: uppercase;
  letter-spacing: 0.4px;
}

tbody tr:nth-child(even) {
  background: #f8fbff;
}

tbody tr:hover {
  background: #eef5ff;
}
```